



Deep Residual Learning for Image Recognition

강유진, 김예나

목차

#01 Introduction

#02 Related Work

#03 Deep Residual Learning

#04 Experiments

#05 Code review



01. Introduction



#01 Introduction

1-1. Why Deeper Networks?

- CNN은 이미지에서 계층적으로 특징을 학습
 - 얇은 층 : edge, texture 등 저수준 특징
 - 깊은 층 : 객체의 부분이나 의미적 특징
 - VGGNet, GoogLeNet 등 당시 성능이 높은 모델들은 깊은 구조를 사용
- ⇒ Network depth가 이미지 인식 성능의 중요한 요소로 여겨짐

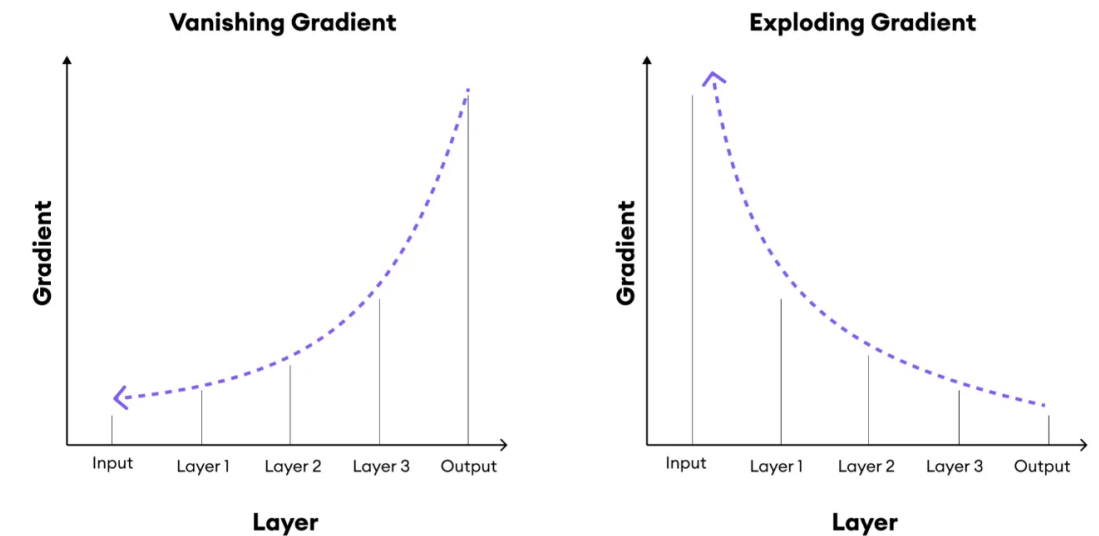
핵심 질문 : 그렇다면 단순히 layer를 계속 추가하면 성능도 계속 좋아질까?

#01 Introduction

1-2. Difficulty of Training Very Deep Networks

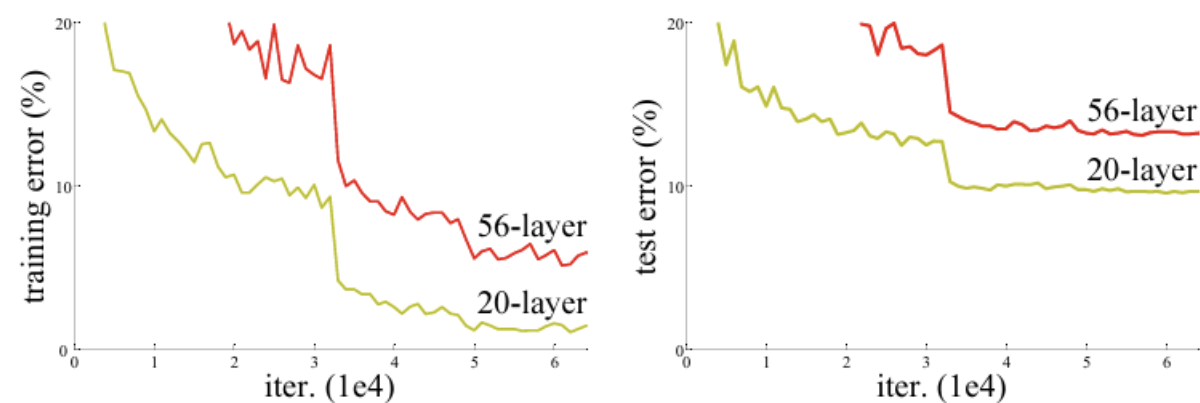
Gradient Vanishing/exploding

- 역전파 과정에서 gradient가 지나치게 작아지거나 커지는 문제
 - 앞쪽 layer가 제대로 학습되지 않거나 학습이 불안정해짐
- => normalized initialization과 Batch Normalization 등으로 상당 부분 해결
- => 이를 통해 수십 개 이상의 layer도 학습 가능해짐



1-3. Degradation Problem

정의 : 충분히 깊은 네트워크에 layer를 추가했을 때, 정확도가 포화되는 것을 넘어 훈련 성능 자체가 나빠지는 현상



- Plain-20보다 Plain-56의 training error가 높음
- test error도 Plain-56이 더 높음
- 깊은 모델이 훈련 데이터조차 충분히 학습하지 못함

Overfitting X, optimization difficulty

#01 Introduction

1-4. Identity Mapping에 관한 사고실험

- 얇은 네트워크에 새로운 layer를 추가한다고 가정
- 추가된 layer가 입력을 그대로 출력하는 identity mapping을 수행하면, 깊은 모델은 최소한 얇은 모델과 동일한 성능을 낼 수 있어야 함

$$H(x) = x$$

⇒ 이론적으로는 깊은 모델의 solution space 안에 얇은 모델의 해가 포함됨

깊은 모델의 training error가 더 높다

= 좋은 해가 존재하지 않는 것이 아니라, 여러 nonlinear layer가 identity mapping을 학습하도록 최적화하기 어렵다

- 이 문제를 해결하기 위해 residual learning을 제안

#01 Introduction

1-5. Paper Overview

1. 깊은 네트워크의 degradation problem을 실험적으로 확인
2. Residual learning framework 제안
3. Parameter-free identity shortcut으로 최적화 개선
4. ImageNet에서 최대 152층, CIFAR-10에서 최대 1202층 모델 학습
5. ImageNet classification 및 다양한 객체 탐지 문제에서 성능 향상

#02 Related Work

2-1 Residual Representations

Residual은 원래 값 전체를 직접 표현하는 대신, 특정 기준과의 차이를 표현하는 방식

기존 이미지 표현 방법

1) VLAD

- 이미지의 local feature와 가장 가까운 dictionary vector 사이의 차이를 계산
- 여러 residual vector를 모아 하나의 이미지 표현 생성

2) Fisher Vector

- 이미지의 local feature가 확률모델의 평균적 특징에서 얼마나 벗어나는지 표현
- VLAD를 확률적으로 확장한 방식으로 해석 가능

=> 이미지 특징 자체보다 특정 기준에서 얼마나 달라졌는지를 표현하는 것이 더 효과적일 수 있음

#02 Related Work

2-2 Residual Based Optimization

Residual을 이용하는 방식은 이미지 인식뿐 아니라 수치해석과 최적화 문제에서도 사용되어 옴

Multigrid method

- 복잡한 문제를 여러 해상도나 scale로 나누어 해결
- 거친 scale에서 구한 근사해와 실제 해 사이의 residual을 다음 단계에서 보정

Hierarchical basis method

- 서로 다른 scale 사이의 차이를 나타내는 residual variable을 사용
- 전체 해를 한 번에 구하기보다 현재 해에서 부족한 부분을 단계적으로 수정

전체 해를 처음부터 직접 찾는 것보다 현재 해와 정답 사이의 차이를 반복적으로 보정하는 방식이 더 빠르게 수렴할 수 있음

-> 문제를 residual 형태로 재정의하면 optimization이 쉬워질 수 있다는 배경으로 제시

#02 Related Work

2-3 Previous Shortcut Connections

초기 MLP 연구

- Network input과 output 사이에 linear connection 추가

Deeply-Supervised Networks

- 중간 layer를 auxiliary classifier와 직접 연결
- 중간 layer까지 supervision과 gradient를 전달

Inception

- 상대적으로 짧은 branch와 깊은 branch를 함께 구성

일부 연구

- Activation, gradient, propagated error를 조절하기 위해 shortcut 형태의 연결 사용

=> ResNet의 핵심은 shortcut을 이용해 다음 형태의 residual function을 명시적으로 학습하도록 구성

#02 Related Work

2-4 Highway Networks

Highway Network는 ResNet과 비슷한 시기에 제안된 대표적인 shortcut 기반 구조

Highway Network의 구조

$$y = H(x) * T(x) + x * C(x)$$

($H(x)$: nonlinear transformation, $T(x)$: transformation gate, $C(x)$: carry gate)

- Gate가 변환된 정보와 기존 정보를 얼마나 통과시킬지 결정

특징

- Gate가 입력 데이터에 따라 정보 흐름을 조절
- Gate에도 학습 가능한 파라미터가 존재
- Gate 값에 따라 shortcut 경로가 열리거나 닫힐 수 있음

#02 Related Work

2-5. Highway Network vs. ResNet

구분	Highway Network	ResNet
Shortcut 제어	학습 가능한 gate	단순 identity mapping
데이터 의존성	입력에 따라 gate가 달라짐	항상 입력을 전달
추가 파라미터	Gate 파라미터 필요	Identity shortcut은 없음
정보 흐름	Gate가 닫힐 수 있음	Shortcut이 항상 열려 있음
학습 대상	Gate를 포함한 변환	Residual function $F(x)$
매우 깊은 모델	논문 당시 제한적 검증	100층 이상까지 효과 검증

핵심 차이

Highway Network에서는 gate가 0에 가까워지면 특정 경로의 정보가 차단될 수 있음
반면 ResNet의 identity shortcut에는 gate가 없으므로 입력 x 가 항상 다음 block으로 전달됨

=> ResNet은 학습 가능한 gate 없이 parameter-free identity shortcut을 사용하여, 더 단순하고 효율적으로 매우 깊은 네트워크 구성

#03 Deep Residual Learning

3-1. Residual Learning

기존 방식의 문제 기존 plain network에서는 여러 개의 layer가 원하는 mapping $H(x)$ 를 직접 학습 | $x \rightarrow H(x)$

x : 여러 layer에 입력되는 feature

$H(x)$: 해당 layer들이 최종적으로 만들어야 하는 출력

But, 네트워크가 깊어지면 여러 nonlinear layer가 $H(x)$ 를 직접 학습하기 어려워지고, training error가 증가하는 degradation problem이 발생

Residual Learning의 핵심 아이디어

ResNet은 원하는 출력 $H(x)$ 전체를 직접 학습하지 않는다. 대신 입력 x 와 원하는 출력 $H(x)$ 의 차이인 residual function을 학습 : $F(x) = H(x) - x$

→ 원래 학습하려던 mapping : $H(x) = F(x) + x$

→ 즉, ResNet의 layer들은 $H(x)$ 전체가 아니라 입력에서 얼마나 변화해야 하는지를 나타내는 $F(x)$ 를 학습

새로운 표현 전체를 처음부터 만드는 대신, 현재 입력을 기준으로 필요한 변화량만 학습

#03 Deep Residual Learning

3-1. Residual Learning

왜 Residual Learning이 더 쉬울까?

- 얇은 네트워크에 layer를 추가해 더 깊은 네트워크를 만든다고 생각
 - 추가된 layer들이 아무런 변환 없이 입력을 그대로 출력하는 identity mapping을 수행한다면, 깊은 모델은 최소한 얇은 모델과 같은 성능을 낼 수 있다. $H(x) = x$
 - 이론적으로는 깊은 모델의 training error가 얇은 모델보다 높아질 이유가 없지만 실제 plain network에서는 깊어질수록 training error가 증가
- ➔ 여러 nonlinear layer가 identity mapping을 직접 학습하는 것이 optimization 과정에서 어렵다

#03 Deep Residual Learning

3-1. Residual Learning

Identity mapping이 최적일 때

기존 plain network : 여러 layer를 통해 $H(x)=x$ 직접 학습해야 한다.

residual learning : residual branch가 $F(x)=0$ 만 학습

최종 출력 : $H(x) = F(x) + x = 0 + x = x$

⇒ 따라서 여러 nonlinear layer가 identity mapping 전체를 직접 학습하는 것보다, residual을 0에 가깝게 만드는 것이 더 쉽다

Identity mapping이 최적이지 아닐 때

실제 상황에서 항상 $H(x)=x$ 인 것은 아니지만 원하는 출력이 현재 입력에서 조금만 변화한 값이라면 $H(x)=x+\text{small modification}$ (작은 수정량 : $F(x)$)

Ex) 원하는 출력이 $1.1x \rightarrow H(x)=1.1x$, Residual Function : $F(x) = H(x)-x = 0.1x$

Plain network는 $1.1x$ 전체를 학습해야 하지만, ResNet은 $0.1x$ 라는 작은 변화만 학습하면 됨

논문의 실험에서도 residual function의 출력값이 일반적으로 작게 나타남 $\rightarrow$ 이는 최적의 mapping이 zero mapping보다 identity mapping에 가깝다는 저자들의 가설을 뒷받침함

#03 Deep Residual Learning

3-2. Identity Mapping by Shortcuts

- Shortcut connection을 이용해 구현
- Residual block의 기본 수식 : $y = F(x, \{W_i\}) + x$

두 개의 layer로 구성된 residual block의 경우 : $F(x) = W_2 \sigma(W_1 x)$ (σ 는 ReLU activation function)
residual branch의 출력과 shortcut branch의 입력을 더하고 ReLU를 적용 $y = \sigma(F(x) + x)$

Residual block의 동작 과정

1. 입력 x 가 residual branch로 전달된다.
2. 여러 weighted layer가 $F(x)$ 를 계산한다.
3. 동시에 shortcut connection이 입력 x 를 그대로 전달한다.
4. $F(x)$ 와 x 를 element-wise addition으로 더한다.
5. 덧셈 결과에 ReLU를 적용한다.

#03 Deep Residual Learning

3-2. Identity Mapping by Shortcuts

Shortcut connection의 특징

1. Identity mapping 수행
2. 추가 파라미터가 없음
3. 계산량이 거의 증가하지 않음
4. 기존 학습 방법을 그대로 사용
5. Plain network와 공정한 비교 가능

Identity shortcut에는 추가 파라미터가 없기 때문에 plain network와 ResNet을 동일한 depth, 동일한 width, 동일한 parameter 수, 거의 동일한 computational cost 등 동일한 조건에서 비교할 수 있다.

따라서 두 모델의 성능 차이를 residual learning의 효과로 해석할 수 있다.

#03 Deep Residual Learning

3-2. Identity Mapping by Shortcuts

입력과 출력의 차원이 다른 경우

$F(x)$ 와 x 를 element-wise로 더하려면 두 feature map의 차원이 같아야 함 (Height, Width, Channel)
CNN에서는 stage가 바뀔 때 Feature map의 공간 크기는 절반으로 감소하고, 채널 수는 2배로 증가하는 등의 변화가 발생

Projection Shortcut

입력과 출력의 차원이 다르면 shortcut에 linear projection W_s 를 적용 : $y = F(x, \{W_i\}) + W_s x$

논문에서는 W_s 를 1x1 convolution으로 구현

1x1 convolution: 채널 수 변경

Stride 2: feature map의 공간 크기 축소

Ex) $W_s x : 56 \times 56 \times 64 \rightarrow 28 \times 28 \times 128$

projection shortcut의 주된 역할은 residual branch와 shortcut branch의 차원을 맞춰 element-wise addition이 가능하게 하는 것

#03 Deep Residual Learning

3-3. Network Architectures: Plain Network

Plain Network

Residual learning의 효과를 비교하기 위한 baseline network로, VGGNet의 설계 원칙을 따름

1) 설계 원칙

- 대부분의 convolution layer에서 3×3 filter 사용
- 같은 크기의 feature map에서는 동일한 filter 수 사용
- Feature map 크기가 절반이 되면 filter 수를 2배로 증가
- Stride=2 convolution으로 downsampling 수행
- 마지막에는 global average pooling과 fully connected layer 사용

- Feature map 변화

$56 \times 56 \times 64 \rightarrow 28 \times 28 \times 128 \rightarrow 14 \times 14 \times 256 \rightarrow 7 \times 7 \times 512$

34-layer plain network의 계산량은 3.6 billion FLOPs

VGG-19의 19.6 billion FLOPs와 비교하면 약 18% 수준

#03 Deep Residual Learning

3-3. Network Architectures: Residual

Network

Residual Network

앞에서 구성한 plain network에 shortcut connection을 추가하여 ㄱ

1) 입력과 출력의 차원이 같은 경우

- Identity shortcut 사용

- $y = F(x) + x$

- 추가 parameter 없음

- 입력과 출력의 차원이 다른 경우

Option A : identity shortcut을 유지하고 증가한 channel을 zero-padding

Option B : 1×1 convolution을 이용한 projection shortcut 사용

Feature map 크기가 변하는 지점에서는 stride=2 적용

Plain-34와 ResNet-34는 depth, width, parameter 수와 계산량이 거의 동일
가장 중요한 차이는 shortcut connection의 존재 여부

=> 이후 실험의 성능 차이를 residual learning에 따른 optimization 효과로 비교 가능

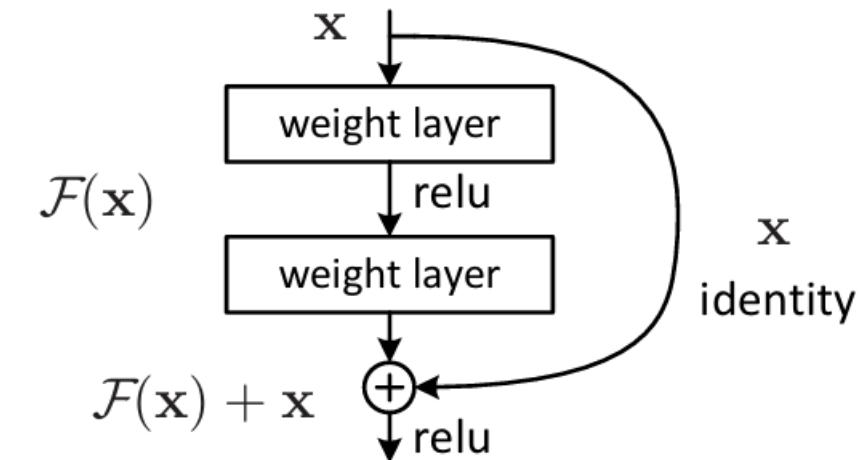


Figure 2. Residual learning: a building block.

#03 Deep Residual Learning

3-3. Basic Block and Bottleneck Block

BasicBlock | ResNet-18 / ResNet-34

3×3 convolution $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ 3×3 convolution $\rightarrow$ BN $\rightarrow$ shortcut 합산 $\rightarrow$ ReLU

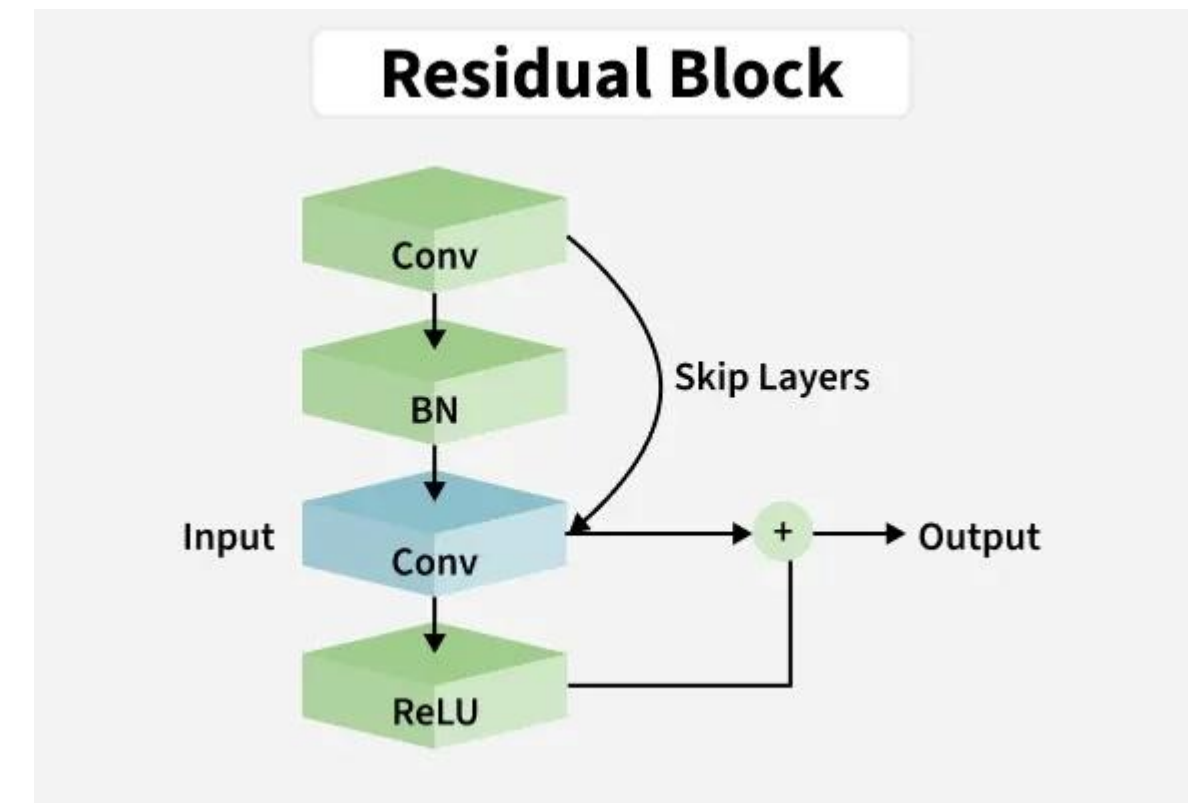
Residual function을 두 개의 3×3 convolution으로 구성

같은 stage에서는 입력과 출력 channel 수를 유지

- Bottleneck Block — ResNet-50 / ResNet-101 / ResNet-152
- 1×1 convolution $\rightarrow$ 3×3 convolution $\rightarrow$ 1×1 convolution
- 첫 번째 1×1 convolution : channel 수 축소
- 3×3 convolution : 줄어든 channel에서 공간 특징 추출
- 마지막 1×1 convolution : channel 수 복원 및 확장

예) $256 \rightarrow 64 \rightarrow 64 \rightarrow 256$

=> 계산량이 큰 3×3 convolution을 적은 channel에서 수행하여, 깊은 network를 효율적으로 구성



#03 Deep Residual Learning

3-4. Implementation

1. Data Preprocessing and Augmentation

- 이미지의 짧은 변을 [256, 480] 범위로 random resize
- 224×224 random crop, horizontal flip
- Per-pixel mean subtraction 및 color augmentation

2. Training

- 모든 convolution 뒤, activation 전에 Batch Normalization 적용
- Plain network와 ResNet 모두 처음부터 학습
- SGD, mini-batch size=256, initial learning rate=0.1
- Momentum=0.9, weight decay=0.0001, dropout 미사용
- Error가 정체되면 learning rate를 1/10로 감소

3. Testing

- 비교 실험에서는 standard 10-crop testing 사용
- 최종 결과는 fully convolutional inference와 multi-scale testing 적용
- 여러 scale에서 얻은 예측 score를 평균

=> 동일한 학습 조건에서 plain network와 ResNet을 비교하여 residual structure의 효과를 검증

#04 Experiments

목차

4.1 ImageNet	degradation 확인 → ResNet으로 해결 → 더 깊은 모델의 성능 향상
A/B/C shortcut	projection이 본질인지, identity shortcut만으로 충분한지 비교
50/101/152 layers	bottleneck으로 매우 깊은 network를 실용적으로 학습
4.2 CIFAR-10	ImageNet 특수 현상이 아니라는 일반성 확인
Layer response	실제 residual branch가 작은 correction처럼 작동하는지 분석
4.3 Detection	classification 밖의 vision task에도 표현이 전이되는지 검증

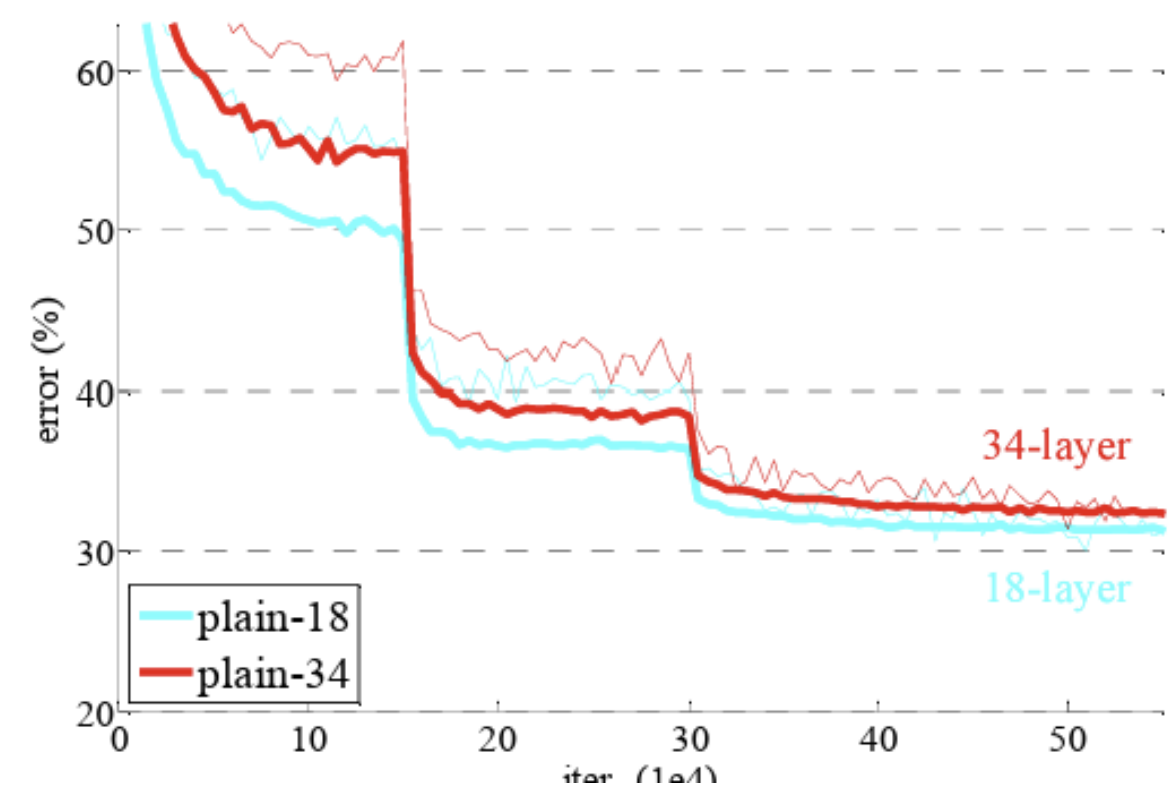
#04 Experiments

ImageNet 실험 세팅과 평가지표

항목	내용	주석
Dataset	ImageNet 2012, 1000 classes	대규모 이미지 분류 benchmark
Train / Val / Test	1.28M / 50k / 100k images	validation은 연구자가 비교, test는 서버 제출
Metric	top-1 error, top-5 error	정답 class가 1위/상위 5개 안에 없는 비율
Testing	10-crop testing	이미지 여러 crop의 예측을 평균해 안정화

#04 Experiments

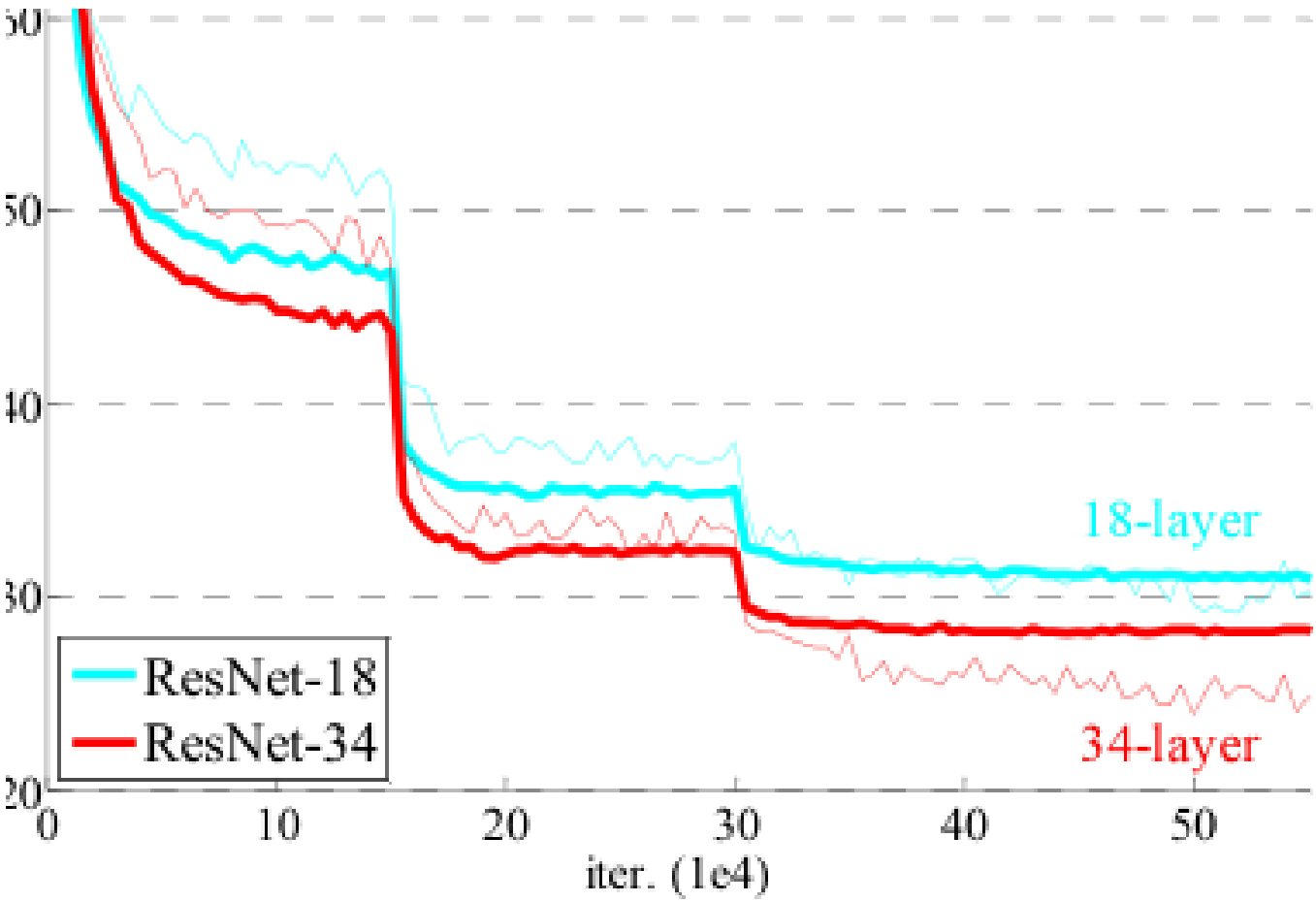
Plain Network



Model	Top-1 error
Plain-18	27.94%
Plain-34	28.54%

#04 Experiments

Plain: depth $\uparrow \rightarrow$ error $\uparrow$ / ResNet: depth $\uparrow \rightarrow$ error $\downarrow$



Model	Top-1 error
ResNet-18	27.88%
ResNet-34	25.03%

#04 Experiments

Shortcut Ablation

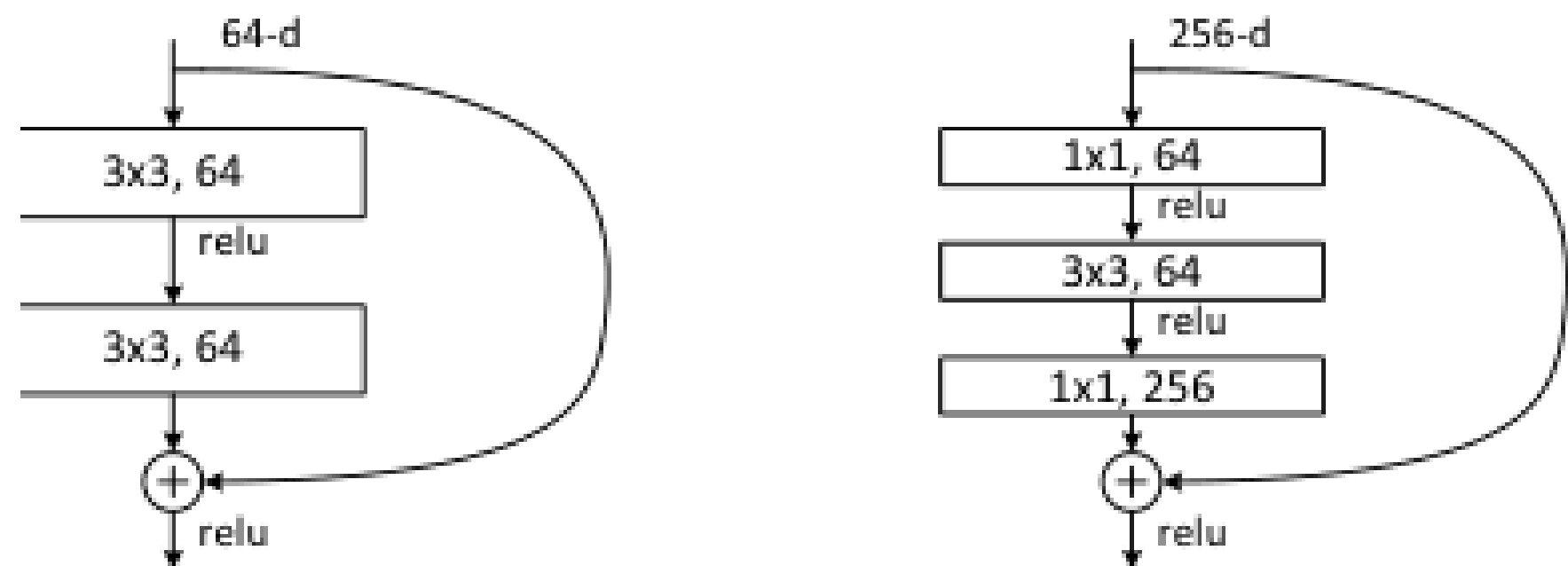
model	top-1 err.	top-5 err.
VGG-16 [40]	28.07	9.33
GoogLeNet [43]	-	9.15
PReLU-net [12]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Option	Shortcut 설계	의미
A	identity + zero-padding	extra parameter 없음
B	차원 증가 시 projection	효율성과 성능 균형
C	모든 shortcut projection	성능 약간 ↑, 비용 ↑

projection이 필수는 아니다.
identity shortcut만으로도 plain-34보다 크게 개선된다.

#04 Experiments

Deeper Bottleneck: 50/101/152



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

is for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks

#04 Experiments

깊이 자체가 문제가 아니라, 깊은 모델을 어떻게 최적화하게 만들 것인가

Model	Top-1 err.	Top-5 err.	비고
ResNet-34 B	24.52	7.46	projection only when dimensions increase
ResNet-50	22.85	6.71	bottleneck
ResNet-101	21.75	6.05	deeper bottleneck
ResNet-152	21.43	5.71	deepest ImageNet model in paper

#04 Experiments

ResNet ensemble: ImageNet test top-5 error 3.57%
→ ILSVRC 2015 classification 1위

method	top-1 err.	top-5 err.
VGG [40] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [43] (ILSVRC'14)	-	7.89
VGG [40] (v5)	24.4	7.1
PRELU-net [12]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

method	top-5 err. (test)
VGG [40] (ILSVRC'14)	7.32
GoogLeNet [43] (ILSVRC'14)	6.66
VGG [40] (v5)	6.8
PRELU-net [12]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	3.57

Table 5. Error rates (%) of ensembles. The top 5 error is on the

Table 4. Error rates (%) of **single-model** results on the ImageNet classification set (percent [†] reported on the test set)

#04 Experiments

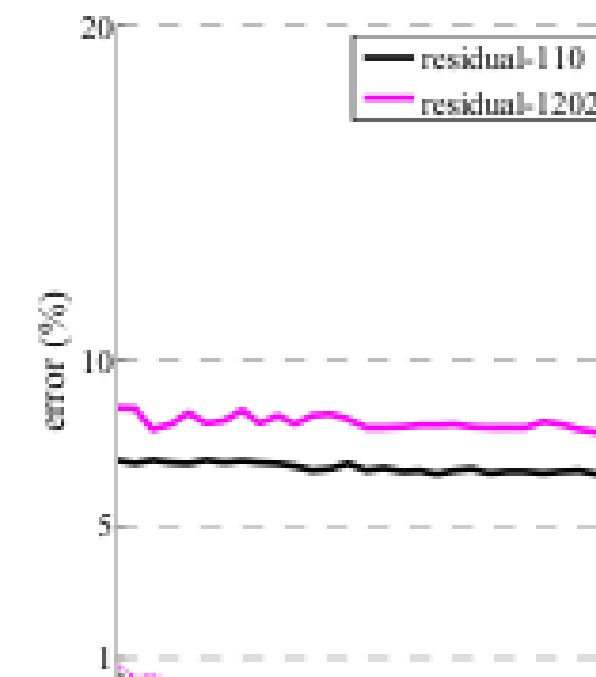
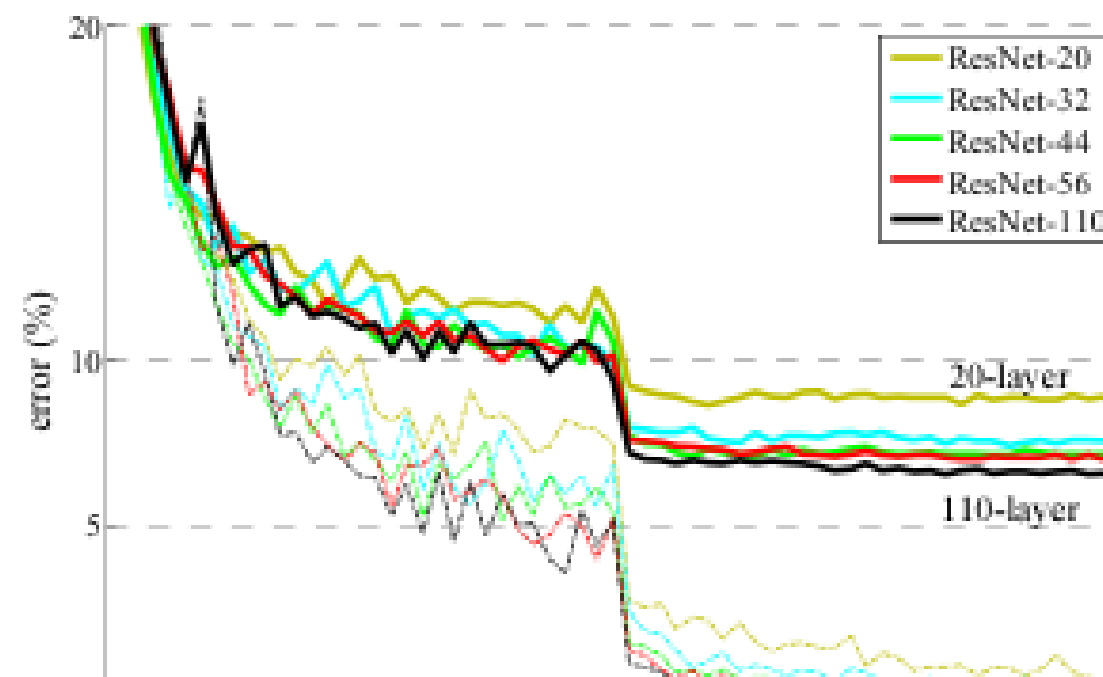
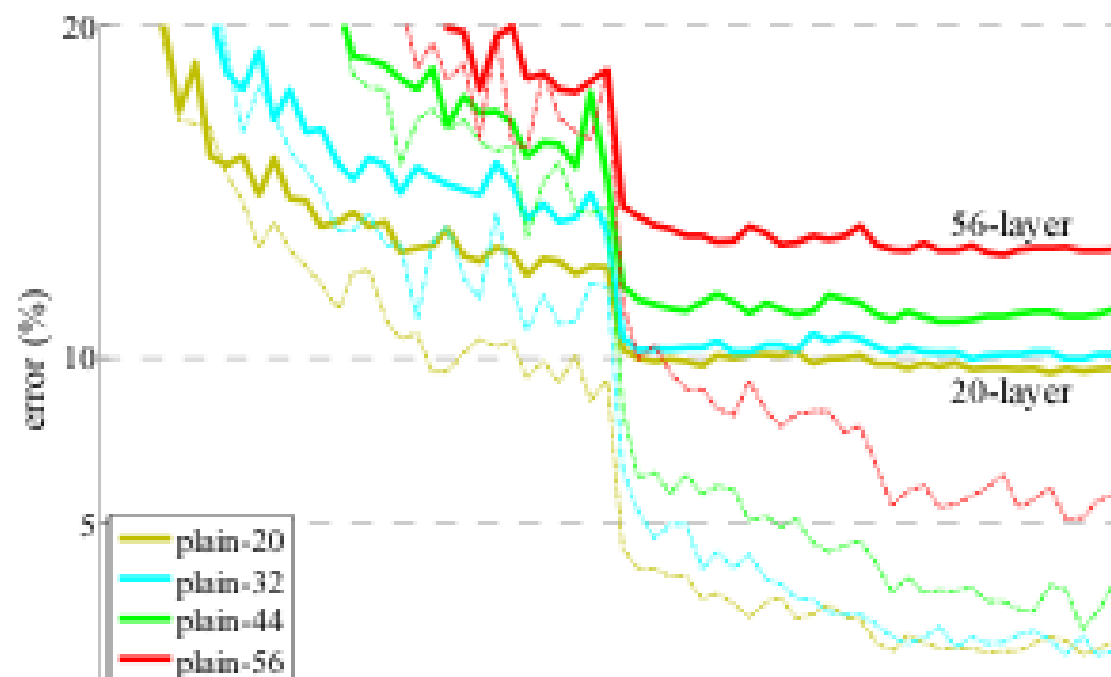
- ResNet은 pair of 3×3 layers마다 shortcut을 연결합니다.
- CIFAR에서는 모든 shortcut을 identity option A로 사용합니다.

항목	내용	주석
Dataset	CIFAR-10	32×32 이미지, 10 classes
Train / Test	50k / 10k images	작은 이미지 분류 benchmark
Architecture	$6n + 2$ weighted layers	n 값으로 depth 조절
Feature maps	$32\times 32 \rightarrow 16\times 16 \rightarrow 8\times 8$	stride 2 conv로 downsampling
Filters	$16 \rightarrow 32 \rightarrow 64$	공간 크기가 줄면 channel 증가

# layers	$1+2n$	$2n$	$2n$
# filters	16	32	64

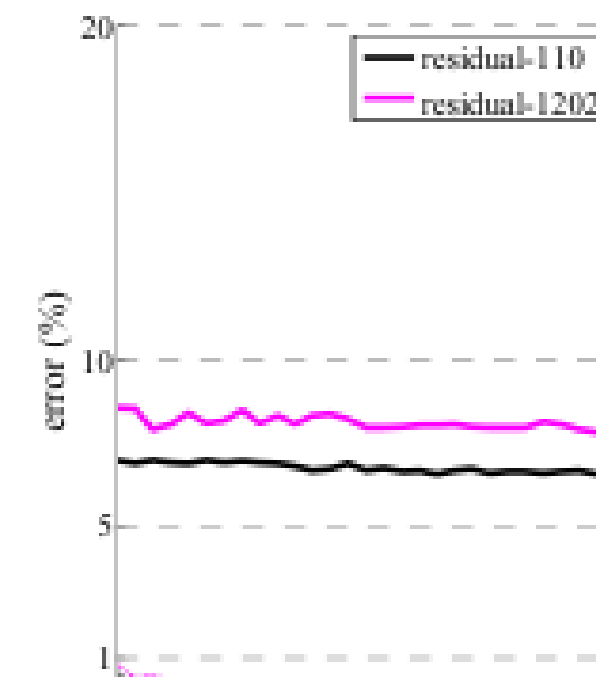
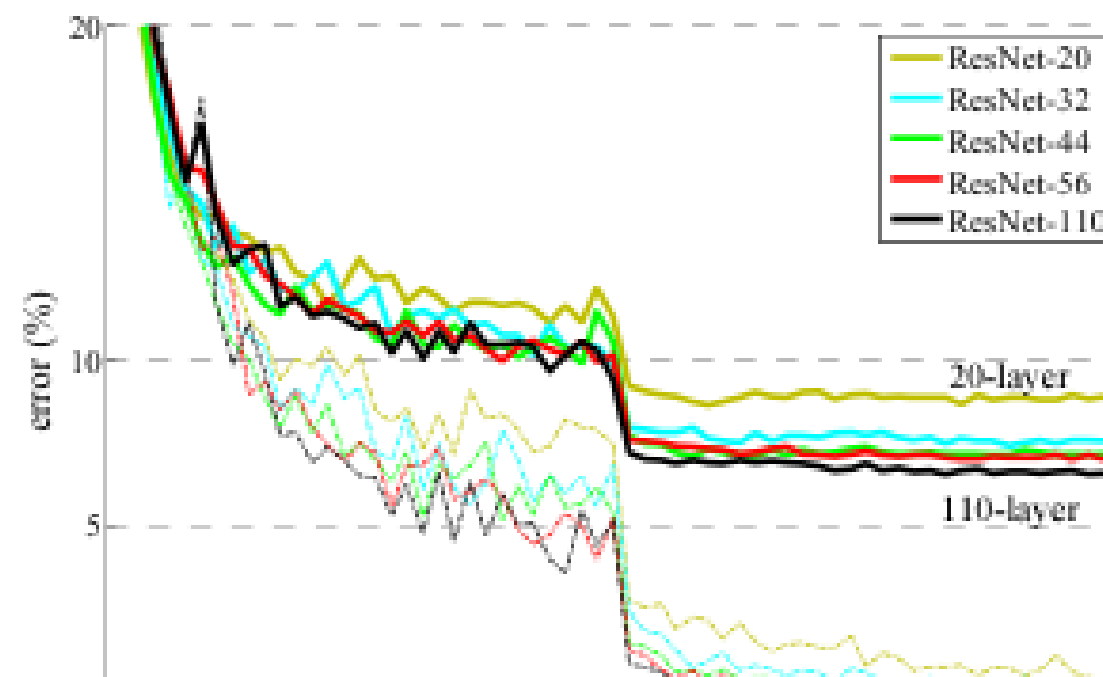
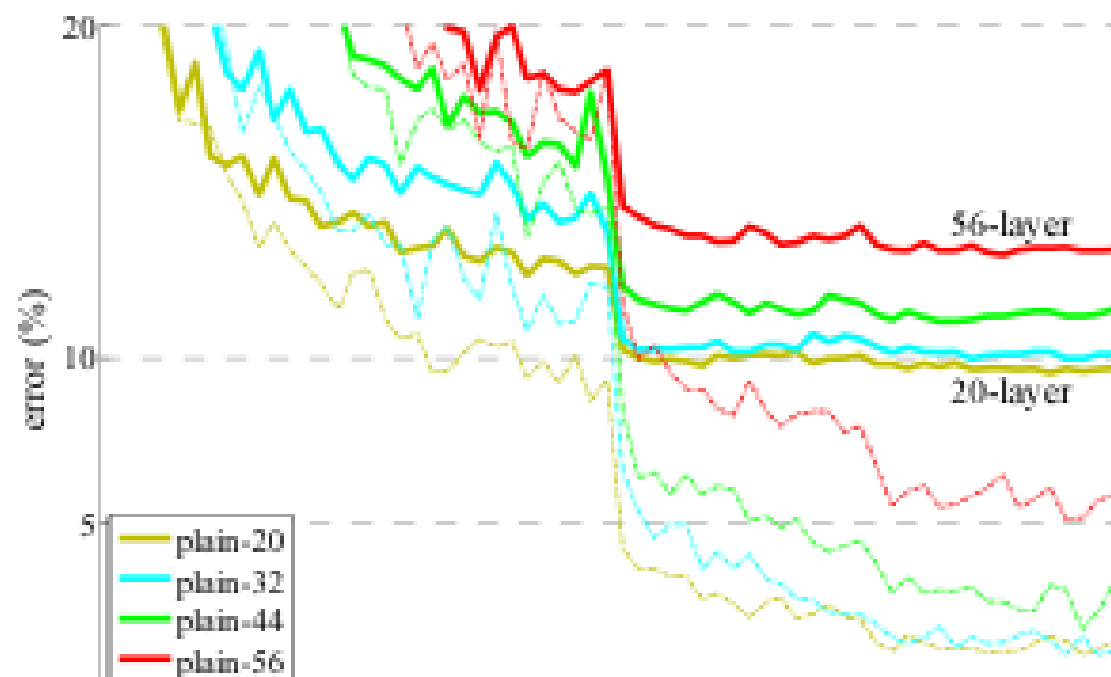
#04 Experiments

- ResNet은 pair of 3×3 layers마다 shortcut을 연결합니다.
- CIFAR에서는 모든 shortcut을 identity option A로 사용합니다.



#04 Experiments

- ResNet은 pair of 3×3 layers마다 shortcut을 연결합니다.
- CIFAR에서는 모든 shortcut을 identity option A로 사용합니다.



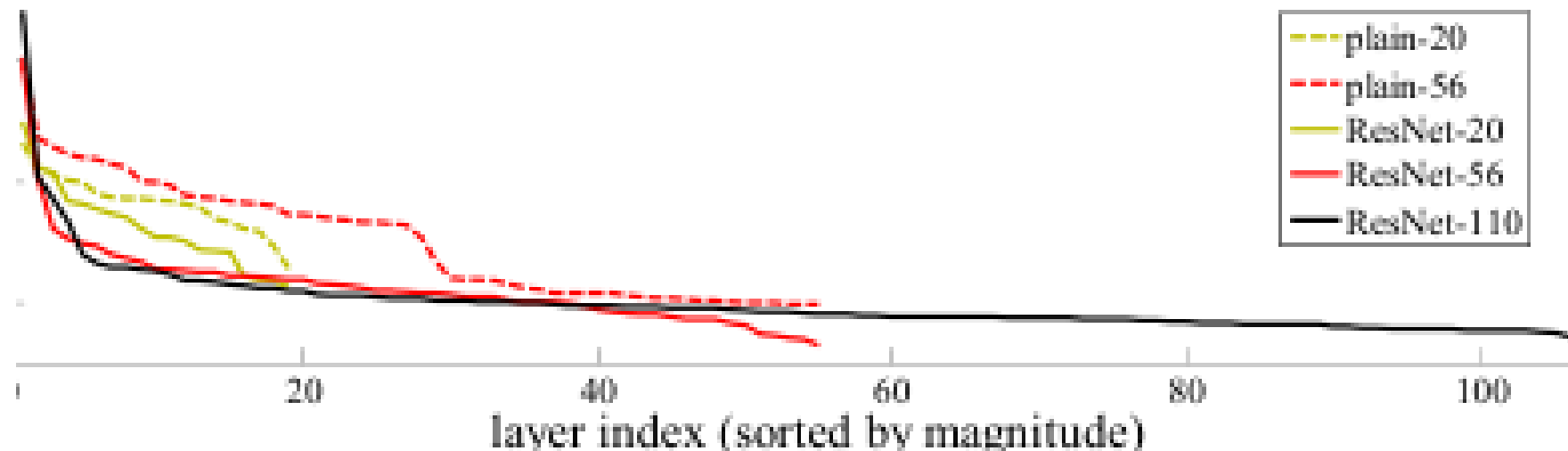
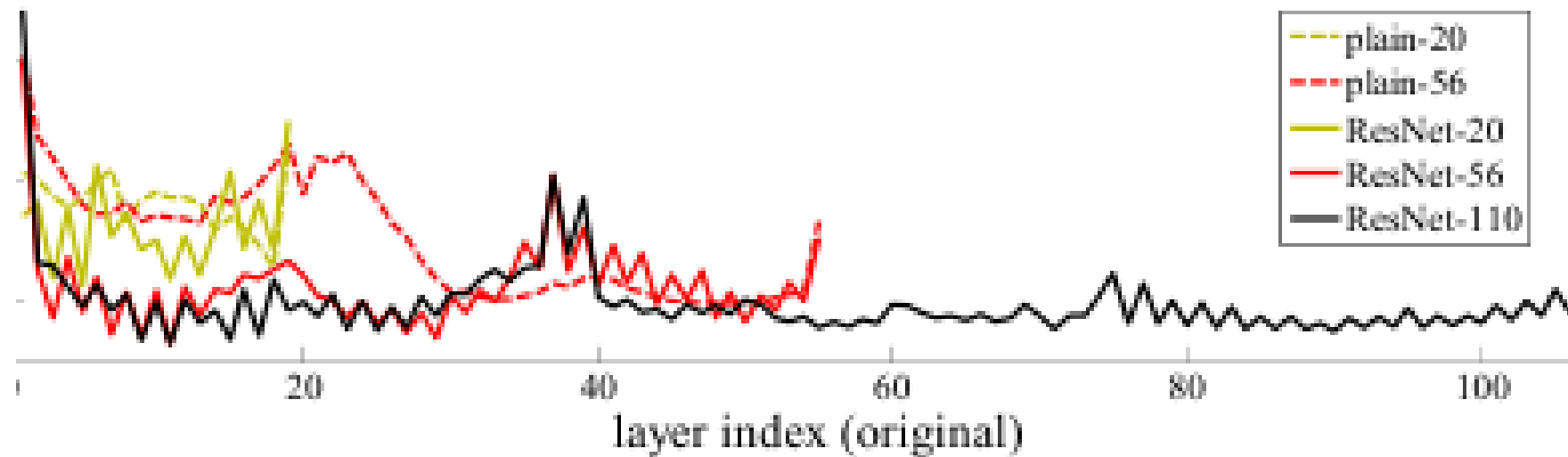
#04 Experiments

- ResNet-110: best error 6.43%, mean 6.61 ± 0.16
- parameter 수: 1.7M으로 FitNet/Highway보다 작거나 비슷합니다.
- 초기 learning rate 0.1이 커서 0.01로 warm-up 후 0.1로 전환합니다.

method			error (%)
Maxout [9]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [34]	19	2.5M	8.39
Highway [41, 42]	19	2.3M	7.54 (7.72 ± 0.16)
Highway [41, 42]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61 ± 0.16)
ResNet	1202	19.4M	7.93

#04 Experiments

- 각 3×3 layer의 response 표준편차(std)를 측정합니다.
- 측정 위치: BN 이후, ReLU/addition 이전입니다.
- ResNet response는 plain network보다 대체로 작습니다.
- 더 깊은 ResNet일수록 개별 layer가 signal을 덜 수정하는 경향이 있습니다.



#04 Experiments : PASCAL

- Task: PASCAL VOC / MS COCO object detection
- Detector: Faster R-CNN baseline
- 비교: VGG-16 backbone vs ResNet-101 backbone
- Detection implementation은 동일하게 두고 backbone만 교체합니다.

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Object detection mAP (%) on the PASCAL

Backbone	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8
Gain	+3.2	+3.4

#04 Experiments : MS COCO

- Task: PASCAL VOC / MS COCO object detection
- Detector: Faster R-CNN baseline
- 비교: VGG-16 backbone vs ResNet-101 backbone
- Detection implementation은 동일하게 두고 backbone만 교체합니다.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Object detection mAP (%) on the COCO validation
baseline Faster R-CNN. See also appendix for better resu

Backbone	mAP@.5	mAP@[.5,.95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2
Gain	+6.9	+6.0

#04 Experiments : 정리

실험	관찰	결론
Plain-18/34	깊은 plain이 training error 증가	degradation 확인
ResNet-18/34	깊은 ResNet이 더 낮은 error	residual learning으로 해결
A/B/C shortcut	identity만으로도 큰 개선	projection이 본질은 아님
50/101/152	깊을수록 성능 향상	depth를 accuracy gain으로 전환
CIFAR-10	동일 패턴 재현	dataset-specific 현상 아님
1202 layers	train은 성공, test는 악화	optimization ≠ generalization
Detection	VGG→ResNet backbone 교체로 mAP ↑	representation의 task transfer

#05 Code Review

[\(1\) resnet.py](#)

```
def conv3x3(in_planes: int, out_planes: int, stride: int = 1, groups: int = 1, dilation: int = 1) -> nn.Conv2d: ...
    )

def conv1x1(in_planes: int, out_planes: int, stride: int = 1) -> nn.Conv2d:
    """1x1 convolution"""
    return nn.Conv2d(in_planes, out_planes, kernel_size=1, stride=stride, bias=False)

class BasicBlock(nn.Module): ...
    return out

class Bottleneck(nn.Module): ...
    return out

class ResNet(nn.Module):| ...
    return self._forward_impl(x)

def _resnet( ...
    return model
```

- conv3x3(), conv1x1()
- BasicBlock
- Bottleneck
- ResNet 클래스
- _make_layer()
- resnet18(), resnet34(),
resnet50() 등 모델 생성
함수

#05 Code Review

(2) conv3x3()와 conv1x1()

```
def conv3x3(in_planes: int, out_planes: int, stride: int = 1, groups: int = 1, dilation: int = 1) -> nn.Conv2d:
    """3x3 convolution with padding"""
    return nn.Conv2d(
        in_planes,
        out_planes,
        kernel_size=3,
        stride=stride,
        padding=dilation,
        groups=groups,
        bias=False,
        dilation=dilation,
    )

def conv1x1(in_planes: int, out_planes: int, stride: int = 1) -> nn.Conv2d:
    """1x1 convolution"""
    return nn.Conv2d(in_planes, out_planes, kernel_size=1, stride=stride, bias=False)
```

- kernel_size=3: 논문에서 기본적으로 사용하는 3×3 convolution
- padding=dilation: feature map 크기 유지
- bias=False: convolution 뒤에 Batch Normalization이 있기 때문에 bias 생략
- stride=2: stage가 바뀔 때 feature map 크기를 절반으로 감소
- Bottleneck에서 channel을 축소.확장
- Shortcut의 입력과 출력 차원을 맞추는 projection shortcut에 사용

#05 Code Review

```
class BasicBlock(nn.Module):
    expansion: int = 1

    def __init__(self, stride: int) -> None:
        self.stride = stride

    def forward(self, x: Tensor) -> Tensor:
        identity = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        if self.downsample is not None:
            identity = self.downsample(x)

        out += identity
        out = self.relu(out)

        return out
```

(3) BasicBlock : ResNet-18과 ResNet-34에서 사용

- identity = x: 입력을 shortcut 경로에 저장
- 두 개의 3×3 convolution으로 F(x) 계산
- out += identity: F(x)+x 구현
- 차원이 다르면 self.downsample(x) 적용
- expansion=1: 출력 channel 수가 planes와 동일

#05 Code Review

```
class Bottleneck(nn.Module):
    # Bottleneck in torchvision places the stride for downsampling at 3x3 convolution(self.conv2)
    # while original implementation places the stride at the first 1x1 convolution(self.conv1)
    # according to "Deep residual learning for image recognition" https://arxiv.org/abs/1512.03385.
    # This variant is also known as ResNet V1.5 and improves accuracy according to
    # https://ngc.nvidia.com/catalog/model-scripts/nvidia:resnet_50_v1_5_for_pytorch.

    expansion: int = 4

    def __init__(
        self,
        inplanes: int,
        planes: int,
        stride: int = 1,
        downsample: Optional[nn.Module] = None,
        groups: int = 1,
        base_width: int = 64,
        dilation: int = 1,
        norm_layer: Optional[Callable[..., nn.Module]] = None,
    ) -> None:
        super().__init__()
        if norm_layer is None:
            norm_layer = nn.BatchNorm2d
        width = int(planes * (base_width / 64.0)) * groups
        # Both self.conv2 and self.downsample layers downsample the input when stride != 1
        self.conv1 = conv1x1(inplanes, width)
        self.bn1 = norm_layer(width)
        self.conv2 = conv3x3(width, width, stride, groups, dilation)
        self.bn2 = norm_layer(width)
        self.conv3 = conv1x1(width, planes * self.expansion)
        self.bn3 = norm_layer(planes * self.expansion)
        self.relu = nn.ReLU(inplace=True)
        self.downsample = downsample
        self.stride = stride
```

(4) Bottleneck

: ResNet-50, ResNet-101, ResNet-152에서 사용

- 첫 번째 1×1 convolution: channel 축소
- 3×3 convolution: 공간적 특징 추출
- 마지막 1×1 convolution: channel 복원 및 확장
- expansion=4: 최종 출력 channel이 planes × 4
- 적은 channel에서 3×3 convolution을 수행해 계산량 절감

#05 Code Review

```
def _make_layer(
    self,
    block: type[Union[BasicBlock, Bottleneck]],
    planes: int,
    blocks: int,
    stride: int = 1,
    dilate: bool = False,
) -> nn.Sequential:
    norm_layer = self._norm_layer
    downsample = None
    previous_dilation = self.dilation
    if dilate:
        self.dilation *= stride
        stride = 1
    if stride != 1 or self.inplanes != planes * block.expansion:
        downsample = nn.Sequential(
            conv1x1(self.inplanes, planes * block.expansion, stride),
            norm_layer(planes * block.expansion),
        )
```

(5)_make_layer()

- 같은 stage에 residual block을 반복해서 쌓는 역할
- stride != 1: feature map 크기가 변하는 경우
- channel 수가 다르면 conv1x1() 사용
- shortcut 경로에도 stride를 적용
- 논문의 projection shortcut, 즉 Option B에 해당
- 이후 첫 번째 block을 만든 다음 나머지 block을 반복

THANK YOU

